Reprendre VeloCite sur un autre PC

Guide Windows PowerShell - le depot a deja ete recupere avec Git

Objectif : reconstruire le cluster Kubernetes local, les environnements dev et prod, puis deployer la version API 2.2.0 avec ses protections de production.

Ce que ce guide reproduit

Le second PC obtiendra la meme architecture et la meme configuration versionnee : cluster kind, outils partages, namespaces, politiques de securite, applications, autoscaler et budgets d'interruption. Les certificats, mots de passe et donnees d'initialisation seront regeneres localement : ils ne doivent pas etre copies depuis le premier PC.

Avant de commencer

Ouvrir PowerShell a la racine du depot. Docker Desktop doit etre demarre et disposer d'au moins 8 Go de memoire (12 Go conseilles). Si PowerShell bloque les scripts, executer une fois :

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Verifier que le depot contient bien les dernieres modifications de l'atelier 6 :

```
git status
git log -1 --oneline
```

Le repertoire de travail ne doit pas contenir de modifications inattendues avant la reprise.

1. Installer les outils du projet

Le script installe les versions attendues de kind, OpenTofu, kubectl et Argo. Il faut fermer puis rouvrir PowerShell apres son execution pour que le PATH soit recharge.

```
./scripts/installer-outils.ps1
# Fermer puis rouvrir PowerShell dans le depot
kind version
tofu version
kubectl version --client
argo version --short
```

2. Definir la phrase de chiffrement OpenTofu

Choisir une nouvelle phrase de passe pour ce second PC et la conserver dans un gestionnaire de mots de passe. Elle chiffre les states locaux d'OpenTofu, qui contiennent des secrets. Ne jamais la committer ni l'ecrire dans un fichier du depot.

```
$env:TF_VAR_phrase_etat = Read-Host "Phrase du state"
```

Cette variable ne vaut que pour la fenetre PowerShell ouverte. Il faut la redefinir apres ouverture d'un nouveau terminal.

3. Liberer le port local et creer le socle

Si Docker Compose a ete utilise sur ce PC, liberer le port 8443. Ensuite, le script cree kind, charge les images VeloCite dans les noeuds, installe les outils du cluster et cree dev.

```
cd app
docker compose down
cd ..
./scripts/up.ps1 -Env dev
kubectl config use-context kind-velocite
kubectl get nodes
```

Resultat attendu : les noeuds velocite-control-plane et velocite-worker sont Ready. Le script peut prendre plusieurs minutes lors du premier lancement.

4. Creer l'environnement de production

Le workspace OpenTofu doit correspondre a la valeur env du fichier prod.tfvars. Ces commandes creent le namespace, les quotas, les politiques reseau et les secrets de prod.

```
tofu -chdir=infra/20-env workspace select -or-create prod
tofu -chdir=infra/20-env plan -var-file=envs/prod.tfvars
tofu -chdir=infra/20-env apply -var-file=envs/prod.tfvars
tofu -chdir=infra/20-env workspace select dev
```

Lire le plan avant de confirmer l'apply. Le retour final au workspace dev evite une commande OpenTofu future lancee par erreur contre prod.

5. Deployer les applications

Les deux overlays utilisent les manifests communs et la version API 2.2.0. En prod, l'API est geree par un HPA (2 a 4 pods), tandis que nginx et ingest ont deux replicas.

Dev :

```
kubectl kustomize deploy/overlays/dev > rendu-dev.yaml
kubectl apply -k deploy/overlays/dev
kubectl -n velocite-dev rollout status deployment/api --timeout=180s
kubectl -n velocite-dev wait --for=condition=complete job/initialisation --timeout=180s
```

Prod :

```
kubectl kustomize deploy/overlays/prod > rendu-prod.yaml
kubectl apply -k deploy/overlays/prod
kubectl -n velocite-prod rollout status deployment/api --timeout=180s
kubectl -n velocite-prod wait --for=condition=complete job/initialisation --timeout=180s
```

Les fichiers rendu-dev.yaml et rendu-prod.yaml sont des fichiers de controle local : ne pas les ajouter a Git.

6. Verifier que les deux environnements sont prets

```
kubectl -n velocite-dev get pods
kubectl -n velocite-prod get pods,hpa,pdb

./scripts/smoke.ps1 -Env dev
./scripts/smoke.ps1 -Env prod

kubectl diff -k deploy/overlays/dev
kubectl diff -k deploy/overlays/prod
```

Resultats attendus

Controle	Attendu
Dev	API 2.2.0, environnement dev, smoke test entierement vert.
Prod	API 2.2.0, environnement prod, smoke test entierement vert.
HPA prod	api, minimum 2 et maximum 4 replicas.
PDB prod	api, ingest et nginx presents, minAvailable = 1.
Diffs	Aucune sortie : le cluster correspond aux manifests Git.

En cas de blocage

Si une commande tofu demande la phrase de state, redefinir TF_VAR_phrase_etat dans le terminal. Si une image est introuvable, relancer ./scripts/charger-images.ps1. Si un pod ne devient pas pret, lire d'abord :

```
kubectl -n velocite-dev get pods
kubectl -n velocite-dev describe pod NOM_DU_POD
kubectl -n velocite-dev logs NOM_DU_POD --previous
```

Pour une mise a jour qui echoue, le service doit rester disponible grace au rollout progressif. Revenir immediatement en arriere avec :

```
kubectl -n velocite-dev rollout undo deployment/api
```

Puis corriger le depot avant de refaire kubectl apply. Ne pas utiliser kubectl edit : la correction serait perdue au prochain apply.